

Build Your First AI-Powered Application

From LLM APIs to RAG — complete code, troubleshooting, and six project briefs. Everything here is free: no credit card, no expiry.

0 Setup

```
python --version # need 3.10+
mkdir ai-workshop && cd ai-workshop
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate

# The "openai" package is a UNIVERSAL client - we are not using OpenAI
pip install openai sentence-transformers numpy streamlit

# Free key, no credit card: https://aistudio.google.com/apikey
export LLM_API_KEY="AIza..." # Windows: setx LLM_API_KEY "AIza..."

# pre-download the local embedding model (~90 MB)
python -c "from sentence_transformers import SentenceTransformer; \
          SentenceTransformer('all-MiniLM-L6-v2')"

mkdir notes # put 3-5 .txt files in here
```

Choosing a provider

The OpenAI request format became the de-facto standard, so almost every provider now speaks it. That means you switch provider by changing **two lines** — nothing else in this handout changes.

PROVIDER	BASE_URL	MODEL	FREE TIER
Google AI Studio default	https://generativelanguage.googleapis.com/v1beta/openai/	gemini-2.5-flash	No credit card, no expiry. Roughly 15 requests/min and 1,500/day on Flash models.
Groq	https://api.groq.com/openai/v1	llama-3.3-70b-versatile	No credit card. Around 30 requests/min. Fastest inference of the three.
Ollama on your laptop	http://localhost:11434/v1	llama3.2	Unlimited, offline, no key (set LLM_API_KEY to any string). Needs ~8 GB RAM.

Model names change every few months. If you get a 404, open your provider's console, look at the current model list, and update the `MODEL` constant. Keeping it in exactly one place at the top of the file is why this is a 10-second fix.

1 The five words

TERM	MEANING
Token	~4 characters. Billed (or rate-limited) per token, input <i>and</i> output. 1,000 tokens ≈ 750 words.
Context window	Everything the model can see in one call. Its entire working memory.
Temperature	0 = deterministic and boring. 1 = creative and risky. Extraction/RAG → 0.
System prompt	Standing rules, sent as the first message every call. Where most of your engineering lives.
Stateless	The API remembers nothing. "Memory" = your app re-sending the transcript.
Grounding	Putting real, retrieved facts in the prompt so answers are checkable. RAG automates this.

2 Lab 1 — first call, then memory

```
# lab1_hello.py
import os
from openai import OpenAI

BASE_URL = "https://generativelanguage.googleapis.com/v1beta/openai/"
MODEL = "gemini-2.5-flash"

client = OpenAI(api_key=os.environ["LLM_API_KEY"], base_url=BASE_URL)

resp = client.chat.completions.create(
    model=MODEL,
    max_tokens=300,          # hard cap on the ANSWER - truncates, does not summarise
    temperature=0.2,        # 0 = repeatable, 1 = creative
    messages=[
        {"role": "system", "content": "You are a concise teaching assistant. "
                                         "Answer in under 60 words. No preamble."},
        {"role": "user", "content": "Explain recursion to a first-year student."},
    ],
)

print(resp.choices[0].message.content)
print("--- in:", resp.usage.prompt_tokens, "out:", resp.usage.completion_tokens)
```

```
# lab1b_chat.py -- multi-turn. YOU are the memory.
SYSTEM = {"role": "system",
          "content": "You are a patient study buddy. Use short paragraphs."}
history = []

while True:
    q = input("\nYou: ")
    if q.lower() in {"quit", "exit"}:
        break
    history.append({"role": "user", "content": q})

    resp = client.chat.completions.create(
```

```

    model=MODEL, max_tokens=500,
    messages=[SYSTEM] + history,          # the WHOLE conversation, every time
)
answer = resp.choices[0].message.content
history.append({"role": "assistant", "content": answer})
print("Bot:", answer)

```

Why `choices[0]` ? The API can return several candidate answers in one call; you asked for one, so it's in slot zero.

Experiments: delete the system message; run at temperature 0 vs 1 three times each; set `max_tokens=20` and notice it gets *cut off* rather than shortened; comment out the `history.append` lines and watch follow-up questions break.

3 Lab 2 — structured output

Prose output needs a human reader. JSON output makes the model a **function in your codebase**: storable, validatable, branchable, chainable, and — crucially — measurable.

```
# lab2_extract.py
import os, json
from openai import OpenAI

BASE_URL = "https://generativelanguage.googleapis.com/v1beta/openai/"
MODEL = "gemini-2.5-flash"
client = OpenAI(api_key=os.environ["LLM_API_KEY"], base_url=BASE_URL)

SYSTEM = """You extract commitments from meeting notes.
Return ONLY valid JSON matching this shape:
{"action_items":[{"task":str,"owner":str,"due":str|null,
                  "priority":"high"|"medium"|"low"}]}

Rules:
- One entry per commitment; never merge two.
- owner must be a name that appears in the notes, or "unassigned". Never invent a person.
- due: ISO date if stated, else null.
- If there are no commitments, return an empty list."""

notes = open("notes/standup.txt", encoding="utf-8").read()

resp = client.chat.completions.create(
    model=MODEL, max_tokens=800, temperature=0,
    response_format={"type": "json_object"},      # <- JSON mode
    messages=[
        {"role": "system", "content": SYSTEM},
        {"role": "user", "content": f"<notes>\n{notes}\n</notes>"},
    ],
)

try:
    data = json.loads(resp.choices[0].message.content)
except json.JSONDecodeError:
    print("Model returned invalid JSON - retry once, then fail loudly")
    raise

for it in data["action_items"]:
    print(f'[{it["priority"]:>6}] {it["owner"]:<12} {it["task"]}')

```

Three things that make JSON reliable: (1) put the exact schema — field names, types, allowed values — in the system prompt; (2) switch on **JSON mode** with `response_format={"type": "json_object"}`, which makes the provider constrain the output so it must parse (Ollama uses `format="json"` instead); (3) still wrap `json.loads` in try/except and retry once — it is a probabilistic system, treat it like a flaky network.

4 Why RAG

APPROACH	HOW	GOOD WHEN	PAIN
----------	-----	-----------	------

Stuff the prompt	Paste the whole document into every call	One small fixed document	Pay for everything, every time; breaks past the context window
Fine-tuning	Retrain weights on your examples	Teaching style/format/skill	1000s of examples, GPUs; facts still go stale and it still hallucinates
RAG	Search first, paste only the top few chunks	Facts that are private, large, or change	You build a search layer — but answers become citable

Fine-tuning teaches the model how to behave. RAG teaches it what is true right now. If your problem is knowledge, you want RAG.

Two phases

Index (once per document): load → chunk → embed → store.

Query (every question): embed the question → cosine top-k → build prompt → generate.

Embeddings and similarity

An embedding model maps text to a vector positioned so that **similar meanings land near each other**. “how do I submit late work?” and “late submission policy” share no keywords but sit almost on top of each other. Two rules you must not break: **use the same embedding model for documents and questions**, and **re-embed everything if you ever switch models**.

We use `all-MiniLM-L6-v2`, which runs **locally on your CPU**: no API key, no quota, no rate limit, no cost. With `normalize_embeddings=True` every vector has length 1, so the dot product is the cosine similarity — your entire vector database is one NumPy array and one matrix multiply. Reach for FAISS, Chroma or pgvector around ~100k chunks, not before.

COSINE SCORE	USUALLY MEANS
> 0.75	Near-restatement — excellent hit
0.45 – 0.75	Related — most useful hits live here
0.25 – 0.45	Loosely on topic — often noise
< 0.25	Unrelated — drop it, and let the app say "I don't know"

Chunking

Too big (a whole chapter): one vector has to represent ten ideas, so retrieval goes vague and you burn tokens. **Too small** (one sentence): you retrieve a fact with no context — “*This is not permitted*” — and the model has to guess what “this” is. **Right:** 100–250 words with ~20% overlap, split on headings/paragraphs before length. When a RAG app is bad, change this *first* — not the model, not the prompt.

5 Lab 3 — the complete RAG app

```
# rag.py -- the whole application
import os, glob, numpy as np
from openai import OpenAI
from sentence_transformers import SentenceTransformer

# ---- change these two lines to switch provider; nothing else changes ----
BASE_URL = "https://generativelanguage.googleapis.com/v1beta/openai/"
MODEL     = "gemini-2.5-flash"

client    = OpenAI(api_key=os.environ["LLM_API_KEY"], base_url=BASE_URL)
embedder  = SentenceTransformer("all-MiniLM-L6-v2") # local, free, no key

# ----- 1. LOAD -----
docs = []
for path in sorted(glob.glob("notes/*.txt")):
    with open(path, encoding="utf-8") as f:
        docs.append((os.path.basename(path), f.read()))

# ----- 2. CHUNK -----
def chunk(text, size=180, overlap=40):
    words, step = text.split(), size - overlap
    return [" ".join(words[i:i + size]) for i in range(0, max(len(words), 1), step)]

chunks = [{"source": name, "text": c} for name, text in docs for c in chunk(text)]
print(f"{len(docs)} docs -> {len(chunks)} chunks")

# ----- 3. EMBED (this is your vector database - runs on your CPU) -----
vectors = embedder.encode([c["text"] for c in chunks],
                           normalize_embeddings=True, show_progress_bar=True)
# np.save("index.npy", vectors) # cache so restarts are instant

# ----- 4. RETRIEVE -----
def retrieve(question, k=3, floor=0.25):
    qv = embedder.encode([question], normalize_embeddings=True)[0]
    scores = vectors @ qv # cosine against every chunk
    order = np.argsort(scores)[::-1][:k]
    return [(chunks[i], float(scores[i])) for i in order if scores[i] >= floor]

# ----- 5. GENERATE -----
SYSTEM = """You answer questions about the user's personal study notes.

Rules:
- Use ONLY the text inside <context>. Never use outside knowledge.
- After each claim, cite the source file in square brackets, e.g. [syllabus.txt].
- If <context> does not contain the answer, reply exactly: "Not in my notes." Do not guess.
- Be concise: 3 sentences maximum."""

def ask(question):
    hits = retrieve(question)
    if not hits:
        return "Not in my notes.", []
    context = "\n\n".join(f"[{h['source']}] {h['text']}" for h, _ in hits)
    resp = client.chat.completions.create(
        model=MODEL, max_tokens=500, temperature=0,
        messages=[
            {"role": "system", "content": SYSTEM},
            {"role": "user", "content": question}
```

```

        f"<context>\n{context}\n</context>\n\nQuestion: {question}",
    ],
)
return resp.choices[0].message.content, hits

if __name__ == "__main__":
    while True:
        answer, hits = ask(input("\nAsk: "))
        print("\n" + answer)
        print("  sources:", ", ".join(h["source"] for h, _ in hits) or "none")

```

Why the `<context>` tags? They draw a hard line between instructions and data. Without them, a document containing “ignore previous instructions” is a live **prompt-injection** hole. Always treat retrieved text as untrusted input.

Test it like an engineer

1. **The easy hit** — something stated almost word-for-word in your notes. Should answer and cite the right file. If it fails, debug `retrieve()`, not the prompt.
2. **The paraphrase** — ask the same thing in completely different words. This is the test keyword search would fail; it's the entire reason you used embeddings.
3. **The trap** — something the model definitely knows but your notes don't contain (“who won the 2011 Cricket World Cup?”). It must say **Not in my notes**. Getting a RAG app to *refuse* is the hard, valuable part.

6 Give it a UI

```

# app.py      run with: streamlit run app.py
import streamlit as st

@st.cache_resource(show_spinner="Indexing your notes...")
def load_app():
    import rag          # imported inside the cache so the index builds ONCE
    return rag

rag = load_app()

st.title("Study Buddy")
st.caption("Answers only from the files in notes/")

q = st.text_input("Ask a question about your notes")
if q:
    with st.spinner("Searching your notes..."):
        answer, hits = rag.ask(q)
        st.markdown(answer)
    with st.expander("Sources used"):
        for h, score in hits:
            st.markdown(f"{h['source']} {score:.2f}")
            st.caption(h["text"][:300] + "...")

```

Streamlit re-runs the whole script on every keystroke, which is why the index build sits inside `@st.cache_resource`. Deploy free on Streamlit Community Cloud — put the API key in **Secrets**, never in the repo.

7 Troubleshooting

API errors

ERROR	REAL CAUSE	FIX
<code>KeyError: 'LLM_API_KEY'</code>	Env var not in this shell / IDE didn't inherit it	Re-export in the same terminal, or use <code>python-dotenv</code>
<code>401 / API key not valid</code>	Typo, trailing space, or key from the wrong project	Re-copy from AI Studio; <code>print(len(key))</code> to catch whitespace
<code>404 / model not found</code>	Model names change every few months	Check your provider's current model list; update <code>MODEL</code>
<code>429 RESOURCE_EXHAUSTED</code>	Free tier is ~15 requests/minute	Wait 60s; add retry with exponential backoff (1s, 2s, 4s, 8s)
<code>400 invalid_request</code>	Two user turns in a row, or empty content	Roles must alternate; content must be non-empty
Answer cut off mid-sentence	<code>max_tokens</code> is a guillotine, not a summariser	Raise it, or ask for brevity in the prompt
<code>response_format</code> rejected	Your provider or model doesn't support JSON mode	Drop it; rely on the schema in the prompt plus <code>try/except</code>
<code>ModuleNotFoundError</code>	Installed into a different Python than you're running	<code>which python</code> — is the venv active?

Bad RAG answers

SYMPTOM	MOST LIKELY CAUSE	CHANGE THIS
Vague, waffly answers	Chunks too large	Drop to 120–180 words
Misses a fact that's clearly there	Fact split across a chunk boundary	More overlap; split on headings first
Says "not in my notes" too often	<code>k</code> too small or floor too high	<code>k=5</code> , floor 0.2, and print the scores
Confidently invents things	Weak grounding rule, or noise chunks passed in	Harden the system prompt; add a similarity floor
Right topic, wrong document	Semantic search ignores exact IDs and codes	Hybrid: BM25 keyword score + cosine
Good chunks, bad answer	Now it really is the prompt or the model	Tighten the prompt; try a larger free model
Painfully slow	Re-embedding on every run	Cache vectors to disk / <code>@st.cache_resource</code>

Diagnostic order, always: print the retrieved chunks → print the exact final prompt → only then blame the model. Most people start at step three and lose a day.

Free tiers, keys and judgement

- Typical free limits are around **15 requests per minute and 1,500 per day**. Plenty for a project; fatal for a runaway loop. Put `break` conditions in retry logic and cache repeated questions.

- **Embeddings cost nothing and hit no quota** — they run on your CPU. Only the generate step spends your allowance, so iterate on chunking and retrieval freely.
- Keys live in environment variables or a secrets manager — never in code, never in a repo. `.gitignore` your `.env` before the first commit. **Never call an LLM API from browser JavaScript**; the key becomes public instantly. Proxy through your own backend.
- **Free tiers are usually free because your prompts may be used to train the model.** Read the terms, then never put anyone's personal data, medical details or confidential documents through one. If the data is sensitive, point `BASE_URL` at Ollama and run offline.
- Show sources, label AI output as AI output, and keep a human in the loop wherever the answer has consequences.

8 Six project briefs

Pick **one**. Give it two evenings. A deployed link and an eval score beat five half-finished repos.

1. Study Buddy+ — chat with a whole semester

LEVEL 1 • NATURAL NEXT STEP

What it does. Upload a semester of PDFs and slides, ask questions across all of them, and generate practice quizzes on any topic — grounded in your syllabus rather than the open internet.

Why it's worth building. It forces you to handle metadata, filtering and incremental re-indexing — the three things every real RAG system needs and every tutorial skips. And it's useful to you in exam week, which means you'll actually finish it.

Stack. `pypdf` · `sentence-transformers` · `NumPy` · `Streamlit` · any free LLM API

Build it:

1. Swap the loader for `pypdf` ; keep page numbers as metadata.
2. Store `{source, page, course}` on every chunk and filter before searching.
3. Hash each file; only re-embed the ones that changed. Cache vectors with `np.save` .
4. Add a quiz mode: retrieve chunks on a topic, then use Lab 2's JSON mode to emit `{question, options, answer, cites}` .
5. Streamlit UI: file uploader, course dropdown, answer panel, sources panel.

Stretch goals:

- Cite page numbers, not just filenames
- “Explain like I'm five” vs “exam-answer format” toggle
- Spaced-repetition flashcards built from questions you got wrong

How to demo it. Ask it something that is *not* in the syllabus and let it refuse. That lands harder than ten correct answers.

2. Campus FAQ Bot

LEVEL 2 • LOOKS MOST PROFESSIONAL

What it does. Answers real student questions — fees, attendance, hostel rules, deadlines — from the official handbook only, citing the exact section, with a clean hand-off to a human when it isn't confident.

Why it's worth building. It's the project on this list that looks like production software: a confidence policy, an escalation path, an audit log and a feedback loop. Reviewers notice those long before they notice your prompt.

Stack. RAG stack • rank_bm25 • SQLite or CSV logging • Streamlit or FastAPI

Build it:

1. Chunk the handbook **by section heading**, storing the section number as metadata.
2. Add hybrid search: BM25 keyword score for codes like “Rule 4.2”, cosine for everything else; merge the two rankings.
3. Enforce a confidence floor — below it, return the relevant office's contact details instead of an answer.
4. Log every question, answer, score and source to a CSV.
5. Read the log weekly. The unanswered questions *are* your roadmap.

Stretch goals:

- An eval set of 30 real questions, 5 of which must be refused — score every change against it
- Thumbs up/down feedback written back to the log
- A weekly digest of the top unanswered questions

How to demo it. Show the log and the eval score, not just the chat window. “28/30, up from 24 after I fixed chunking” is an engineering sentence.

3. Resume ↔ Job Description Matcher

LEVEL 1 • EMBEDDINGS, NO LLM

What it does. Paste a job description and your CV. It scores overall fit, shows which requirements are well evidenced, and flags the ones you've said nothing about — then optionally drafts honest bullets to close the gaps.

Why it's worth building. There is **no LLM in the core loop at all** — just a matrix of cosine similarities, which means it costs nothing and hits no rate limit. It proves embeddings are a tool in their own right and makes the maths of retrieval concrete in a way a chatbot never does.

Stack. sentence-transformers • NumPy • matplotlib • Streamlit

Build it:

1. Split the JD into one requirement per line and the CV into one bullet per line.
2. Embed both lists; compute `sim = R @ B.T` (requirements × bullets).
3. For each requirement take `sim.max(axis=1)` — the best evidence you have for it.
4. Anything below ~0.35 is a gap. Report covered / weak / missing.
5. Only now call an LLM, once per gap: “write one honest bullet I could earn.”

Stretch goals:

- Heatmap of requirements × bullets
- Batch mode: rank 20 job posts against one CV
- Tune the threshold against 10 hand-labelled examples

How to demo it. Show the heatmap. Frame it as “*find my honest gaps*”, never “*beat the ATS*” — the second one writes you a CV you can't defend in an interview.

4. Notes → Tasks microservice

LEVEL 2 • BACKEND FLAVOUR

What it does. Lab 2 wrapped in a real API: POST messy meeting notes, GET back validated JSON tasks with owners, due dates and priorities. Optionally pushes them into Notion, Trello or a database.

Why it's worth building. You learn the part that turns a script into software: schema validation, error handling, retries, and what to do when the model returns something malformed at 2am with no human watching.

Stack. FastAPI • Pydantic • SQLite • pytest

Build it:

1. Define the output as a Pydantic model — that model is now your contract.
2. One endpoint: `POST /extract` taking raw text.
3. Validate the model's JSON against the schema; on failure, retry once with the validation error appended to the prompt, then return a clean 422.
4. Handle `429` from the free tier with exponential backoff, and queue requests rather than dropping them.
5. Write pytest cases — including notes with zero commitments and deliberately ambiguous ones.

Stretch goals:

- Async + batching for multiple notes
- A tiny React or Streamlit front end
- A usage counter: tokens and requests per day against your free quota

How to demo it. Hit it with `curl` live and show the 422 on a deliberately broken input. Graceful failure is the demo.

5. Codebase Explainer

LEVEL 3 • HARDEST CHUNKING PROBLEM

What it does. RAG over a GitHub repository. “Where is authentication handled?” → the answer, plus the files and line ranges to open.

Why it's worth building. Code breaks every naive chunking assumption: a function split in half is useless, and a function without its imports is nearly as bad. Solving that teaches you more about retrieval than ten text-based projects.

Stack. Python `ast` module · tree-sitter (multi-language) · RAG stack

Build it:

1. Walk the repo; skip `.git`, `node_modules`, binaries and lockfiles.
2. Chunk **by function and class** using `ast` — never by word count.
3. Prepend a header to each chunk: file path, class name, signature. That context is what makes retrieval work.
4. Store start/end line numbers so answers can point at real locations.
5. System prompt: answer with file paths and line ranges; refuse if the code isn't in context.

Stretch goals:

- Include the README and docs in the same index
- “Explain this diff” mode fed from `git diff`
- Call-graph awareness: pull in the functions a retrieved function calls

How to demo it. Point it at a repo you've never seen and answer a question about it faster than anyone in the room can find it by hand.

6. Research Paper Digest

LEVEL 3 • MAP-REDUCE + RAG

What it does. Drop in 10–30 PDFs. Get a structured summary of each (problem, method, dataset, result, limitation) in a sortable table, then ask questions across the whole collection.

Why it's worth building. It combines both halves of today: structured extraction per document, then retrieval across all of them. It also teaches **map-reduce summarisation** — the standard pattern for anything longer than a context window.

Stack. pypdf or GROBID • JSON mode • RAG stack • pandas

Build it:

1. Extract text per paper; keep section headings where you can find them.
2. **Map:** summarise each section separately into the fixed JSON schema.
3. **Reduce:** merge section summaries into one paper-level record.
4. Watch your rate limit here — 30 papers × 6 sections is 180 calls. Batch, cache, and sleep between them.
5. Index the full text separately for cross-paper questions like “which papers used ImageNet?”

Stretch goals:

- Cluster papers by embedding similarity to spot research themes
- Auto-generate a related-work paragraph with real citations
- Flag contradictions between papers

How to demo it. Show the comparison table first, the chat second. The table is what makes people say “I would use that.”

9 What to do this week

1. **Tonight (20 min).** Point today's app at a folder that matters to you and ask it ten real questions. Write down every wrong answer.
2. **Tomorrow (1 hr).** Turn those ten into `evals.csv`. Change one thing (chunk size), re-run, watch the score move. **This is the skill that gets you hired** — anyone can demo, almost nobody measures.
3. **This weekend.** Add the Streamlit UI and deploy it. A live link beats a screenshot.
4. **Next week.** Pick one brief above and give it two evenings.

One habit that compounds: keep a `prompts.md` in every AI project. Every time you fix a prompt, paste the before/after and one line on why. In three months it will be the most valuable file in your repo.

Where to read next

- [ai.google.dev](#) — Gemini API docs: prompting, structured output, tool use, rate limits.
- [console.groq.com/docs](#) — if you want faster free inference on open models.
- [ollama.com](#) — running models entirely offline on your own machine.
- [sbert.net](#) — embeddings, semantic search, rerankers.
- [Chroma](#) / [pgvector](#) / [Qdrant docs](#) — for when a NumPy array stops being enough.
- [Streamlit docs](#) — caching, session state, deployment.

Don't start with LangChain. You've now written the primitives by hand. Pick up a framework when you feel the specific pain it solves — you'll still be able to debug it when it breaks.